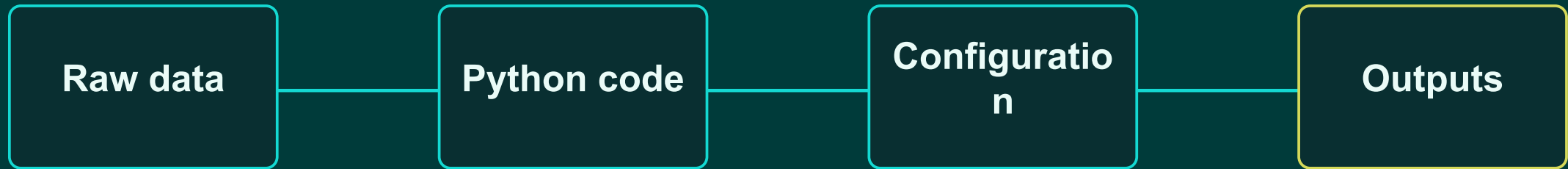


# Organizing a Complete Python Data Project

From working scripts to repeatable, understandable project structure

Day 5 Block 8 • 30 min teaching + 30 min exercises



Project organization is how a program explains itself before anyone reads the code.

# Today's Big Move

The same program becomes more maintainable when each part has a home.

```
processor.py  
records.csv  
output.csv  
test_stuff.py  
notes.txt  
config.json
```

everything mixed together



```
project/  
├── main.py  
├── src/  
├── tests/  
├── data/  
└── config/
```

separated by purpose

Mental model: folders are labels for responsibility.

# Why “One Folder With Everything” Breaks Down

Small scripts are easy to start, but hard to trust as they grow.

Hard to know which file starts the program

Raw input and generated output get mixed together

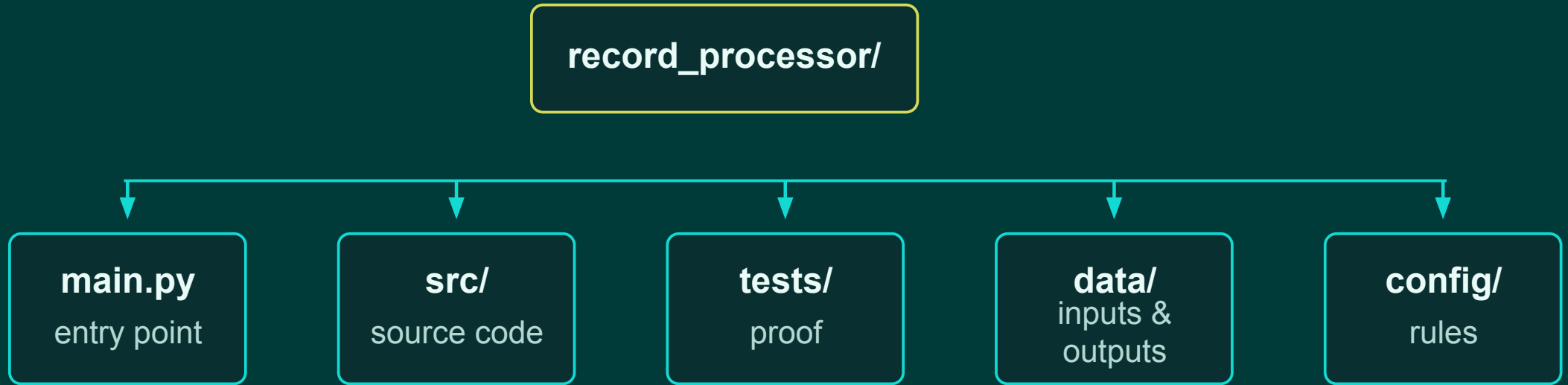
Tests, notes, and source code blur together

Re-running safely becomes risky

Organization is not decoration. It is part of making software reliable.

# Project Structure: Separation by Purpose

A project folder should tell a reader where to look.



The layout is not universal. The principle is universal: separate by purpose.

# A Clean Day 5 Project Layout

This is the target structure students will build.

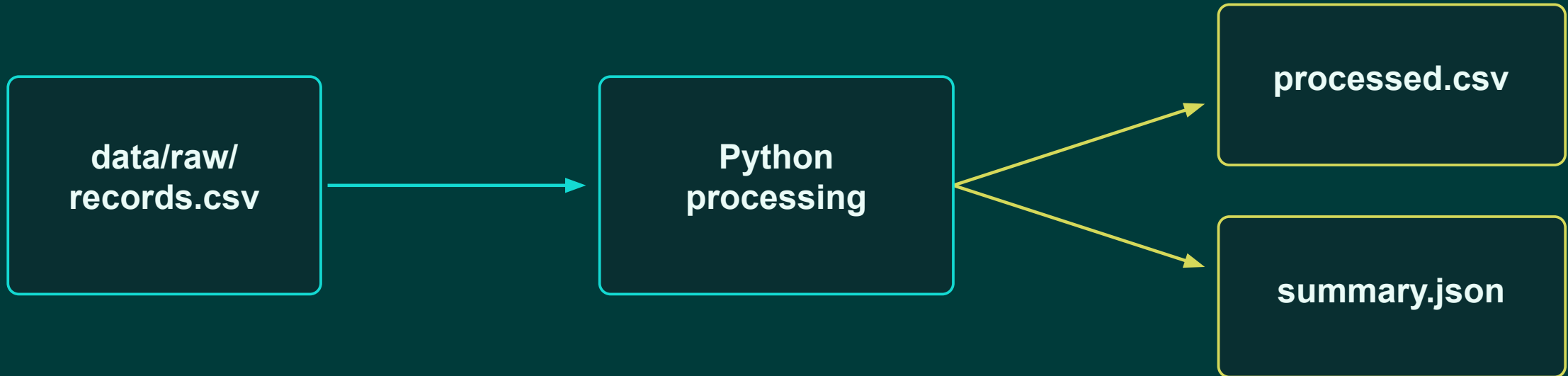
```
record_processor/  
├── main.py  
├── requirements.txt  
  
├── src/  
│   ├── file_io.py  
│   ├── validation.py  
│   ├── transformations.py  
│   └── reporting.py  
  
├── data/  
│   ├── raw/  
│   │   └── records.csv  
│   └── output/  
  
├── config/  
│   └── processing_config.json  
├── tests/  
│   └── test_validation.py
```

## Read it like a map

main.py shows the workflow  
src/ contains reusable logic  
data/raw/ is source input  
data/output/ is generated  
tests/ prove behavior

## Raw Data Is Treated as Read-Only

Keep original source files separate from generated files.



Good habit: outputs can be deleted and regenerated; raw data should be preserved.

# Output Data Should Be Regenerable

If the program is repeatable, output is a product—not the source of truth.

Raw data  
+ code  
+ configuration  
-----  
Same output



## Why this matters

Recover from mistakes  
Compare before and after  
Debug with the same inputs  
Show how results were produced

This is the beginning of reproducible data processing.

## Source Code: Group Related Logic

The src/ folder is where reusable program behavior lives.

**file\_io.py**

Load and write files

**validation.py**

Check data rules

**transformations.  
py**

Normalize and  
classify

**reporting.py**

Create summaries

Not one function per file. One responsibility per file.

```
from src.validation import validate_record
from src.transformations import normalize_record
from src.reporting import create_summary
```



# Tests: Put Proof Beside the Program

tests/ contains automated checks that support future changes.

```
src/  
└─ validation.py  
  
def  
is_valid_completion(value)  
:  
    return 0 <= value <=  
100
```



```
tests/  
└─ test_validation.py  
  
def test_completion_100():  
    assert is_valid_completion(100)
```

Production logic + tests proving expected behavior.

## Configuration: Rules That Should Be Easy to Change

Configuration files keep thresholds and allowed values out of scattered code.

```
{  
  "maximum_errors": 5,  
  "allowed_statuses": [  
    "active",  
    "inactive"  
  ]  
}
```



```
maximum_errors = (  
    config["maximum_errors"]  
)  
  
if error_count > maximum_errors:  
    status = "rejected"
```

Changing a business rule should not require hunting through ten functions.

# Documentation and Dependencies Have a Place Too

A project is more than .py files.

**requirements.txt**

Third-party packages needed to run the project

**README.md**

How to set up, run, and understand the project

**config/**

Project settings that change behavior

**tests/**

Evidence the project still behaves correctly

Reproducibility depends on code, data, configuration, and setup instructions.

# Avoid Hard-Coded Absolute Paths

Absolute paths often work only on one computer.

```
file_name = (  
    "/home/mike/python101/"  
    "data/raw/equipment.csv"  
)
```

## Problem

- Depends on one username
- Depends on one folder layout
- Breaks when project moves

A project should move from one machine to another without editing every path.

# Construct Paths Intentionally with pathlib

Build paths from known project locations instead of raw strings.

```
from pathlib import Path

project_root = Path(__file__).parent

input_file = (
    project_root
    / "data"
    / "raw"
    / "equipment.csv"
)
```

## Key idea

Paths should be based on  
the project, not on one  
person's computer.

The exact root strategy can vary; the  
habit matters.

# Entry Point: Make the Starting Place Obvious

main.py coordinates the workflow.

```
def main():  
    print("Starting pipeline")  
  
    records = load_records()  
  
    processed_records = (  
        process_records(records)  
    )  
  
    write_results(processed_records)  
  
    print("Processing complete")  
  
if __name__ == "__main__":  
    main()
```

## main.py answers

Where does the program start?  
What happens first?  
What order do stages run?  
Where are details delegated?

## main() Should Read Like an Outline

The details live in smaller functions and modules.



A good main function lets the reader understand the process before seeing implementation details.

# Naming Conventions Reduce Guesswork

Names should help a reader predict what belongs where.

## Vague

`test.py`

`stuff.py`

`final_results2.csv`

`new_new_processor.py`

## Purposeful

`test_validation.py`

`transformations.py`

`processed_records.csv`

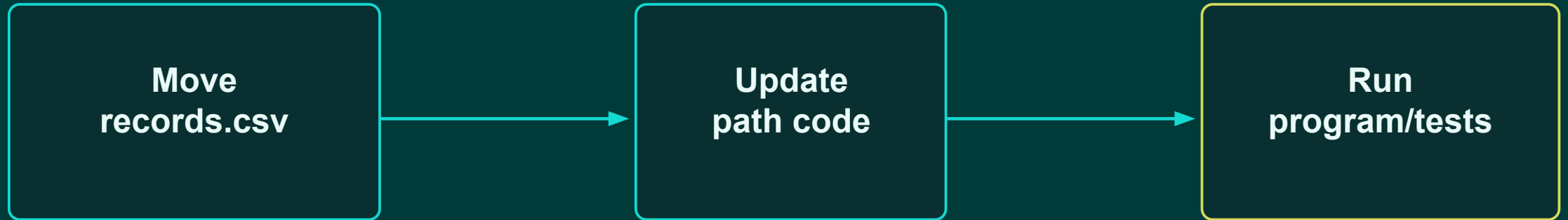
`main.py`

File names are part of your communication with future readers.



## Common Mistake: Moving Files Without Updating Code

Organization changes paths and imports. The code must agree with the structure.



A clean folder tree is only finished when the project still runs.

## Guided Exercise: Clean Up a Messy Project

Students start with a working but disorganized folder.

```
project/  
  everything.py  
  records.csv  
  records_clean.csv  
  test.py  
  config.json  
  notes.txt
```

### Task

- Identify each file's purpose
- Propose a cleaner structure
- Move files carefully
- Run the project again

Exercise time: 30 minutes total

## Exercise Step 1: Sort by Purpose

Before moving files, decide what each file is for.

**Entry point**

`main.py`

**Source logic**

`src/`

**Raw data**

`data/raw/`

**Generated output**

`data/output/`

**Configuration**

`config/`

**Tests**

`tests/`

Do not move first and think later. Categorize first.

## Exercise Step 2: Build the Target Folder Tree

Create the structure before updating code.

```
project/
├── main.py
├── requirements.txt
├── src/
├── tests/
├── data/
│   ├── raw/
│   └── output/
└── config/
```

### Instructor prompt

Which folder should contain  
files you are allowed to  
regenerate?

Expected: data/output/

Timing: 7 minutes

## Exercise Step 3: Update Paths and Imports

Moving files changes how Python finds things.

```
# Before  
with open("records.csv") as file:  
    ...
```

```
# Before  
def validate_record(record):  
    ...
```

```
# After  
from pathlib import Path  
  
project_root = Path(__file__).parent  
input_file = (  
    project_root / "data" / "raw" /  
    "records.csv"  
)
```

```
# After  
from src.validation import (  
    validate_record,  
)
```

Timing: 8 minutes

## Exercise Step 4: Run, Verify, and Explain

A reorganization is successful only when the workflow still works.

Program starts from main.py  
Raw data is read from data/raw/  
Output appears in data/output/  
Config is read from config/  
Tests still run  
No accidental raw-data overwrite

### Student explanation

“Where would I look if  
validation logic was wrong?”

Expected: src/validation.py and tests/

Timing: 7 minutes

# Applied Challenge: Build a Small Production-Style Project

Students combine everything from Day 5 into one organized application.

```
record_processor/  
├── main.py  
├── requirements.txt  
├── src/  
│   ├── file_io.py  
│   ├── validation.py  
│   ├── transformations.py  
│   └── reporting.py  
├── data/  
│   ├── raw/  
│   │   └── records.csv  
│   └── output/  
├── config/  
│   └── processing_config.json  
└── tests/  
    └── test_validation.py
```

## Challenge goal

A complete file-based application with a clear starting point, reusable modules, separated data folders, configuration, and tests.

Use the morning CSV + JSON pipeline as the starting program.

## Required Module Responsibilities

Each module should have a clear job.

### `file_io.py`

- Loading CSV
- Loading JSON
- Writing CSV
- Writing JSON

### `validation.py`

- Schema validation
- Allowed values
- Range checks

### `transformations.py`

- Normalization
- Type conversion
- Classification

### `reporting.py`

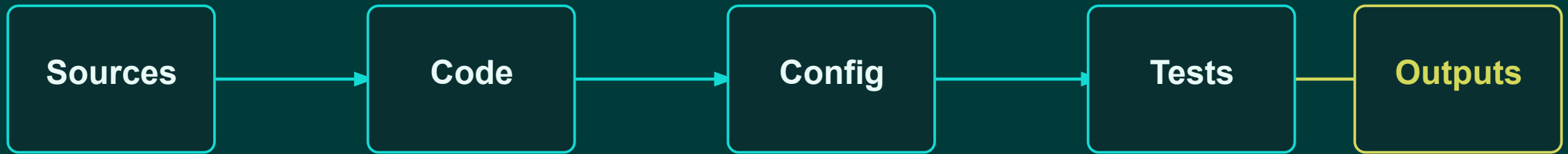
- Summary metrics
- Output summary
- Counts

`main.py` should call these pieces in the correct order—not contain all the details.



## Block 40 Wrap-Up

By the end, students should see a Python program as a small data system.



Organized projects make programs easier to run, easier to test, easier to debug, and easier to hand to someone else.

Next: Day 6 moves from manual CSV loops into dataframe-style tabular processing.